

Introduction to Data Science and Engineering

- Reshaping data and joining tables



Some materials courtesy of
Rafael A. Irizarry, and are modified
from the original version.

Zhenqin (Michael) Wu / 吳楨欽

School of Computing and Data Science
University of Hong Kong

Slide deck originally created by RB Luo



In this lecture

- Data wrangling challenges
- Tidy, long, and wide
- Joining & binding
- Set operators in R



In this lecture

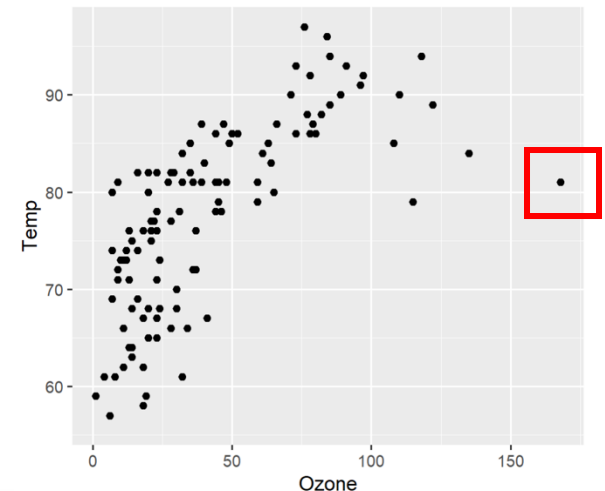
- **Data wrangling challenges**
- Tidy, long, and wide
- Joining & binding
- Set operators in R

Purpose of data wrangling

- We have been using organized and tidy data: e.g., `murders`
- In real life applications, data usually comes “dirty”, especially when data are extracted from web pages, tweets, and/or PDFs. In these cases, the first step is to import and **clean** the data.
 - To annotate and handle missing entries
 - To remove outliers and incorrect data
 - To standardize format
 - ...

```
> data("airquality")
> mean(airquality$Ozone)
[1] NA
> sum(is.na(airquality$Ozone))
[1] 37
```

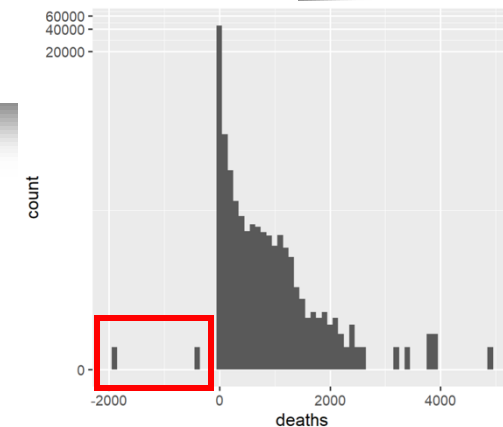
```
> airquality |> ggplot(aes(Ozone, Temp)) + geom_point()
Warning message:
Removed 37 rows containing missing values or values outside the scale range (`geom_point()`).
```



Purpose of data wrangling

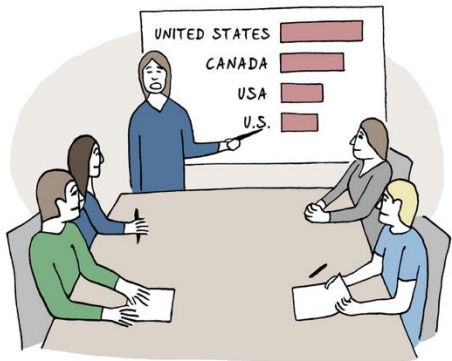
- We have been using organized and tidy data: e.g., `murders`
- In real life applications, data usually comes “dirty”, especially when data are extracted from web pages, tweets, and/or PDFs. In these cases, the first step is to import and **clean** the data.
 - To annotate and handle missing entries
 - To remove outliers and incorrect data
 - To standardize format
 - ...

```
> covid_data <- read.csv("http://www.bio8.cs.hku.hk/comp2501/covid.csv")
> covid_data |>
+   ggplot(aes(deaths)) +
+   geom_histogram(binwidth = 100) +
+   scale_y_continuous(transform = "log1p")
```



Purpose of data wrangling

- We have been using organized and tidy data: e.g., `murders`
- In real life applications, data usually comes “dirty”, especially when data are extracted from web pages, tweets, and/or PDFs. In these cases, the first step is to import and **clean** the data.
 - To annotate and handle missing entries
 - To remove outliers and incorrect data
 - To standardize format
 - ...



AS YOU CAN SEE, OUR TOP MARKETS ARE
UNITED STATES, CANADA, USA AND THE U.S.

```
> data("reported_heights")
> unique(reported_heights$height)[1:50]
[1] "75"      "70"      "68"      "74"      "61"
[6] "65"      "66"      "62"      "67"      "72"
[11] "6"       "69"      "64"      "60"      "5' 4\""
[16] "73"      "71"      "66.75"   "5.3"     "63"
[21] "70.5"    "165cm"   "511"     "64.1732" "68.5"
[26] "2"       "69.6"    "5'7"     "66.5"    "71.5"
[31] "76"      "74.5"    "78"      ">9000"    "5'7\""
[36] "62.5"    "5'3\""   "77"      "68.89"   "64.173"
[41] "59"      "67.7"    "71.7"    "70.87"   "5 feet and 8.11 inches"
[46] "5.25"    "64.57"   "51"      "5'11"    "5.5"
```


Purpose of data wrangling

- We have been using organized and tidy data: e.g., murders
- In real life applications, data usually comes “dirty”, especially when data are extracted from web pages, tweets, and/or PDFs. In these cases, the first step is to import and **clean** the data.





Purpose of data wrangling

- We have been using organized and tidy data: e.g., `murders`
- In real life applications, data usually comes “dirty”, especially when data are extracted from web pages, tweets, and/or PDFs. In these cases, the first step is to import and **clean** the data.
 - To annotate and handle missing entries
 - To remove outliers and incorrect data
 - To standardize format
 - ...
- We will cover several common steps of the data wrangling/cleaning process:
 - Data table operations
 - String processing
 - Date/time processing
 - Web page (html) parsing



In this lecture

- Data wrangling challenges
- **Tidy, long, and wide**
- Joining & binding
- Set operators in R



Review: tidy data tables

After importing data, a common next step is to organize the data into a form that facilitates the rest of the analysis: a tidy format

A data table is **tidy** if:

- each row represents **ONE** observation;
- each column represent a different variable available for observation

```
> library(tidyverse)
> library(dslabs)
> path <- system.file("extdata", package="dslabs")
> filename <- file.path(path, "fertility-two-countries-example.csv")
> wide_data <- read_csv(filename)
```

Rows: 2 Columns: 57

— Column specification —

Delimiter: ","

chr (1): country

dbl (56): 1960, 1961, 1962, 1963, 1964, 1965, 1966, 1967, 1968, 1969,...

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

Review: tidy data tables

After importing data, a common next step is to organize the data into a form that facilitates the rest of the analysis: a tidy format

A data table is **tidy** if:

- each row represents **ONE** observation;
- each column represent a different variable available for observation

```
> wide_data
# A tibble: 2 × 57
  country    `1960` `1961` `1962` `1963` `1964` `1965` `1966` `1967` `1968`
  <chr>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Germany    2.41  2.44  2.47  2.49  2.49  2.48  2.44  2.37  2.28
2 South Korea 6.16  5.99  5.79  5.57  5.36  5.16  4.99  4.85  4.73
# ... with 47 more variables: `1969` <dbl>, `1970` <dbl>, `1971` <dbl>,
#   `1972` <dbl>, `1973` <dbl>, `1974` <dbl>, `1975` <dbl>
#   `1977` <dbl>, `1978` <dbl>, `1979` <dbl>, `1980` <dbl>
#   `1982` <dbl>, `1983` <dbl>, `1984` <dbl>, `1985` <dbl>
#   `1987` <dbl>, `1988` <dbl>, `1989` <dbl>, `1990` <dbl>
#   `1992` <dbl>, `1993` <dbl>, `1994` <dbl>, `1995` <dbl>
#   `1997` <dbl>, `1998` <dbl>, `1999` <dbl>, `2000` <dbl>
# i Use `colnames()` to see all variable names
```





Wide vs Long vs Tidy

Aspect	Long Format	Wide Format	Tidy Format
Definition	Each row is an observation; fewer columns.	Variables are spread across columns.	Each variable is a column, each row an observation.
Structure	Narrow and tall (many rows).	Wide and short (many columns).	Standardized for analysis tools.
Use Case	Analysis, plotting (e.g., time-series).	Reporting, human readability.	Data analysis workflows.

Long table is not necessarily tidy:

- There could be multiple rows pointing to the same observation, but containing different variables
- No requirements for unique identifiers

Wide -> Long: `pivot_longer`

- We want to reshape the fertility dataset so that each row represents **ONE** fertility observation (one country & one year): we need three columns to store the year, country, and the observed value.
- More generally, we are splitting one row into multiple rows
 - Which columns are we splitting?
 - How to rename them?

	country	1960	1961	1962	1963	1964	1965	1966
1	Germany	2.41	2.44	2.47	2.49	2.49	2.48	2.44
2	South Korea	6.16	5.99	5.79	5.57	5.36	5.16	4.99

```
> new_tidy_data <- pivot_longer(  
+   wide_data,  
+   `1960`:`2015`,  
+   names_to = "year",  
+   values_to = "fertility",  
+ )
```



	country	year	fertility
1	Germany	1960	2.41
2	Germany	1961	2.44
3	Germany	1962	2.47
4	Germany	1963	2.49
5	Germany	1964	2.49
6	Germany	1965	2.48
7	Germany	1966	2.44



What happened? backticks

If not using backticks, 1960:2015 is interpreted as a sequence

```
> new_tidy_data <- pivot_longer(wide_data, 1960:2015, names_to = "year", values_to = "fertility")
Error in `build_longer_spec()` :
! Can't subset columns past the end.
i Locations 1960, 1961, 1962, ..., 2014, and 2015 don't exist.
i There are only 57 columns.
Run `rlang::last_error()` to see where the error occurred.
```

- ``1960`:`2015`` means any columns from column named 1960 to column named 2015, including all columns in between them;
- `' '` and `" "` also work, but R standard prefers using ```
- `` `` allows space. If a column is called "life expectancy", use ``life expectancy`` in `pivot_longer`

```
> new_tidy_data <- pivot_longer(wide_data, '1960':'2015', names_to = "year", values_to = "fertility")
> new_tidy_data <- pivot_longer(wide_data, "1960":"2015", names_to = "year", values_to = "fertility")
```



What happened?

names_to & values_to

	country	1960	1961	1962	1963	1964	1965	1966
1	Germany	2.41	2.44	2.47	2.49	2.49	2.48	2.44
2	South Korea	6.16	5.99	5.79	5.57	5.36	5.16	4.99

names_to
values_to

	country	year	fertility
1	Germany	1960	2.41
2	Germany	1961	2.44
3	Germany	1962	2.47
4	Germany	1963	2.49
5	Germany	1964	2.49
6	Germany	1965	2.48
7	Germany	1966	2.44



A more convenient way: indexing with negative sign

Specify which column will not include in the pivot

```
> new_tidy_data <- wide_data |>  
+ pivot_longer(-country, names_to = "year", values_to = "fertility")
```

- Indexing with negative number (`[-1]`) gives the same data structure, but with the specified item excluded
- Same logic here, use negative sign to indicate which column(s) to exclude from pivoting



Use correct data types

Is it all? Let's check the data frame

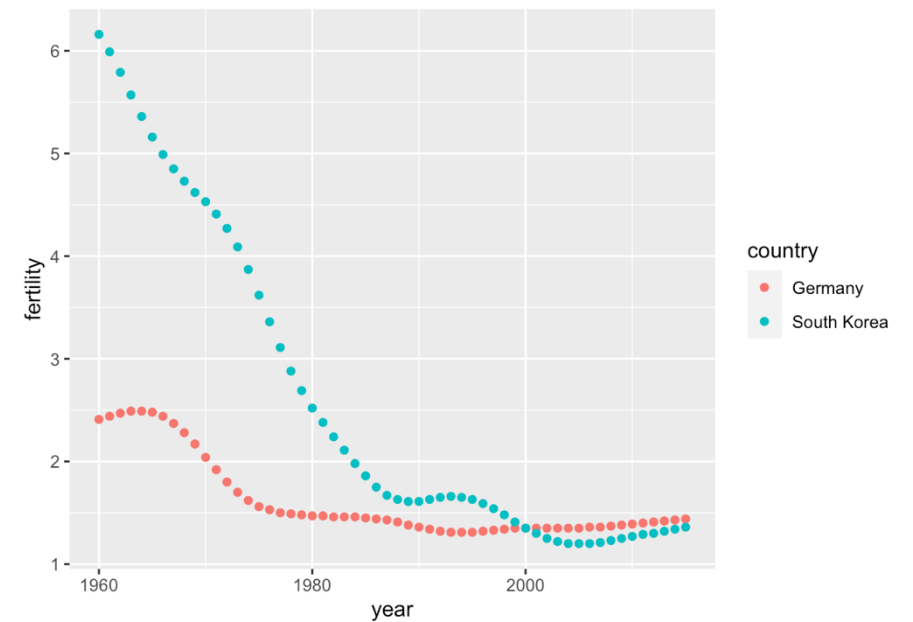
```
> str(new_tidy_data)
tibble [112 × 3] (S3: tbl_df/tbl/data.frame)
 $ country  : chr [1:112] "Germany" "Germany" "Germany" "Germany" ...
 $ year     : chr [1:112] "1960" "1961" "1962" "1963" ...
 $ fertility: num [1:112] 2.41 2.44 2.47 2.49 2.49 2.48 2.44 2.37 2.28 2.17
 ...
```

`pivot_longer` assumes that column names are characters To facilitate subsequent analyses, we need to convert the year column to be numbers:

```
> new_tidy_data <- wide_data |>
+ pivot_longer(-country, names_to = "year", values_to = "fertility") |>
+ mutate(year = as.integer(year))
```

Plot with tidy data

```
> new_tidy_data |>  
+ ggplot(aes(year, fertility, color = country)) +  
+ geom_point()
```

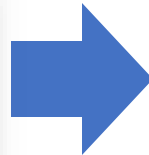


Long -> Wide: `pivot_wider`

There are also scenarios where we want to convert long data into wide data: when a long table contains multiple rows for one observation.

`pivot_wider` is the inverse of `pivot_longer`

```
> new_wide_data <- pivot_wider(  
+   new_tidy_data,  
+   names_from = "year",  
+   values_from = "fertility",  
+ )
```



```
> new_wide_data  
# A tibble: 2 x 57  
  country `1960` `1961` `1962` `1963` `1964` `1965` `1966` `1967` `1968`  
  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  
1 Germany  2.41  2.44  2.47  2.49  2.49  2.48  2.44  2.37  2.28  
2 South ... 6.16  5.99  5.79  5.57  5.36  5.16  4.99  4.85  4.73  
# ... with 47 more variables: `1969` <dbl>, `1970` <dbl>, `1971` <dbl>,  
# `1972` <dbl>, `1973` <dbl>, `1974` <dbl>, `1975` <dbl>,  
# `1976` <dbl>, `1977` <dbl>, `1978` <dbl>, `1979` <dbl>,  
# `1980` <dbl>, `1981` <dbl>, `1982` <dbl>, `1983` <dbl>,  
# `1984` <dbl>, `1985` <dbl>, `1986` <dbl>, `1987` <dbl>,  
# `1988` <dbl>, `1989` <dbl>, `1990` <dbl>, `1991` <dbl>,  
# `1992` <dbl>, `1993` <dbl>, `1994` <dbl>, `1995` <dbl>, ...  
# i Use `colnames()` to see all variable names
```

- Now we are combining multiple rows into one row, things could be tricky:
 - Two (long data) rows will be combined if (and only if) ALL other columns are the SAME.
 - All unique values in the `names_from` column of the long data table will be treated as columns for the wide data table

Long -> Wide: pivot_wider

```
> head(murders, 3)
  state abb region population total
1 Alabama AL  South    4779736   135
2  Alaska AK   West     710231    19
3 Arizona AZ   West    6392017   232
> murders |>
+   select(region, state, population) |>
+   pivot_wider(names_from = "state", values_from = "population")
```

```
# A tibble: 4 x 52
  region Alabama Alaska Arizona Arkansas California Colorado Connecticut Delaware `District of Columbia` Florida Georgia Hawaii
  <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
1 South    4779736    NA      NA    2915918    NA      NA      NA      897934    601723  1.97e7  9920000    NA
2 West      NA    710231  6392017    NA    37253956  5029196    NA      NA      NA    NA    NA      NA    1360301
3 Northeast    NA      NA      NA      NA      NA      NA      NA    3574097    NA    NA      NA      NA
4 North Centr...    NA      NA      NA      NA      NA      NA      NA      NA    NA    NA      NA      NA
# i 39 more variables: Idaho <dbl>, Illinois <dbl>, Indiana <dbl>, Iowa <dbl>, Kansas <dbl>, Kentucky <dbl>, Louisiana <dbl>,
# Maine <dbl>, Maryland <dbl>, Massachusetts <dbl>, Michigan <dbl>, Minnesota <dbl>, Mississippi <dbl>, Missouri <dbl>,
# Montana <dbl>, Nebraska <dbl>, Nevada <dbl>, `New Hampshire` <dbl>, `New Jersey` <dbl>, `New Mexico` <dbl>, `New York` <dbl>,
# `North Carolina` <dbl>, `North Dakota` <dbl>, Ohio <dbl>, Oklahoma <dbl>, Oregon <dbl>, Pennsylvania <dbl>, `Rhode Island` <dbl>,
# `South Carolina` <dbl>, `South Dakota` <dbl>, Tennessee <dbl>, Texas <dbl>, Utah <dbl>, Vermont <dbl>, Virginia <dbl>,
# Washington <dbl>, `West Virginia` <dbl>, Wisconsin <dbl>, Wyoming <dbl>
```

- Now we are combining multiple rows into one row, things could be tricky:
 - Two (long data) rows will be combined if (and only if) ALL other columns are the SAME.
 - All unique values in the `names_from` column of the long data table will be treated as columns for the wide data table
 - Missing entries will be filled by NAs

How about multiple variables stored in a wide(r) format?

```
> path <- system.file("extdata", package = "dslabs")
> filename <- "life-expectancy-and-fertility-two-countries-example.csv"
> filename <- file.path(path, filename)
> raw_dat <- read_csv(filename)
Rows: 2 Columns: 113
— Column specification —————
Delimiter: ","
chr   (1): country
dbl  (112): 1960_fertility, 1960_life_expectancy, 1961_fertility, 196...
```

i Use ``spec()`` to retrieve the full column specification for this data.
i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

	country	1960_fertility	1960_life_expectancy	1961_fertility	1961_life_expe
1	Germany	2.41	69.26	2.44	
2	South Korea	6.16	53.02	5.99	

A decorative abstract graphic on the left side of the slide, consisting of multiple overlapping, wavy, grey lines that create a sense of movement and depth.

A quick try with `pivot_longer`:

the table got transformed but now with one observation spanning two rows, we need to combine them (how?).

```
> raw_dat |> pivot_longer(-country)
# A tibble: 224 x 3
  country name variable      value
  <chr>   <chr>   <dbl>
1 Germany 1960_fertility 2.41
2 Germany 1960_life_expectancy 69.3
3 Germany 1961_fertility 2.44
4 Germany 1961_life_expectancy 69.8
5 Germany 1962_fertility 2.47
6 Germany 1962_life_expectancy 70.0
7 Germany 1963_fertility 2.49
8 Germany 1963_life_expectancy 70.1
9 Germany 1964_fertility 2.49
10 Germany 1964_life_expectancy 70.7
# ... with 214 more rows
# i Use `print(n = ...)` to see more rows
```

Ideally, we want to
separate another
“variable” column and
perform `pivot_wider`

separate

The `separate` function takes three arguments: the name of the column to be separated, the names to be used for the new columns, and the separator (character that separates the variables).

```
> raw_dat |>  
+ pivot_longer(-country) |>  
+ separate(name, c("year", "name"), "_")  
# A tibble: 224 x 4
```

	country	year	name	value
	<chr>	<chr>	<chr>	<dbl>
1	Germany	1960	fertility	2.41
2	Germany	1960	life	69.3
3	Germany	1961	fertility	2.44
4	Germany	1961	life	69.8
5	Germany	1962	fertility	2.47
6	Germany	1962	life	70.0
7	Germany	1963	fertility	2.49
8	Germany	1963	life	70.1
9	Germany	1964	fertility	2.49
10	Germany	1964	life	70.7

```
# ... with 214 more rows
```

```
# i Use `print(n = ...)` to see more rows
```

Warning message:

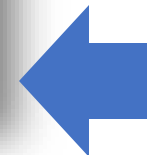
Expected 2 pieces. Additional pieces discarded in 112 rows [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, ...].

You got "life" instead
of "life_expectancy"
(why?)

separate

The problem doesn't solve if you just keep specifying more columns

```
> raw_dat |>
+ pivot_longer(-country) |>
+ separate(name, c("year", "name1", "name2"), "_")
# A tibble: 224 x 5
  country year  name1    name2    value
  <chr>   <chr> <chr>    <chr>    <dbl>
1 Germany 1960 fertility NA        2.41
2 Germany 1960 life     expectancy 69.3
3 Germany 1961 fertility NA        2.44
4 Germany 1961 life     expectancy 69.8
5 Germany 1962 fertility NA        2.47
6 Germany 1962 life     expectancy 70.0
7 Germany 1963 fertility NA        2.49
8 Germany 1963 life     expectancy 70.1
9 Germany 1964 fertility NA        2.49
10 Germany 1964 life     expectancy 70.7
# ... with 214 more rows
# i Use `print(n = ...)` to see more rows
Warning message:
Expected 3 pieces. Missing pieces filled with `NA` in 112 rows [1, 3, 5, 7,
9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39, ...].
```



separate

```
> raw_dat |>
+ pivot_longer(-country) |>
+ separate(name, c("year", "name"), extra = "merge")
# A tibble: 224 x 4
  country year  name      value
  <chr>   <chr> <chr>    <dbl>
1 Germany 1960 fertility 2.41
2 Germany 1960 life_expectancy 69.3
3 Germany 1961 fertility 2.44
4 Germany 1961 life_expectancy 69.8
5 Germany 1962 fertility 2.47
6 Germany 1962 life_expectancy 70.0
7 Germany 1963 fertility 2.49
8 Germany 1963 life_expectancy 70.1
9 Germany 1964 fertility 2.49
10 Germany 1964 life_expectancy 70.7
# ... with 214 more rows
# i Use `print(n = ...)` to see more rows
```

Try `?separate`, it supports handling of multiple variables through arguments `extra` and `fill`.

A better solution is to use regular expression. We will see how to do that in the next lecture.

separate + pivot_wider

	country	1960_fertility	1960_life_expectancy	1961_fertility	1961_life_expe
1	Germany	2.41	69.26	2.44	
2	South Korea	6.16	53.02	5.99	

```
> raw_dat |>
+ pivot_longer(-country) |>
+ separate(name, c("year", "name"), extra = "merge") |>
+ pivot_wider()
# A tibble: 112 x 4
  country year  fertility life_expectancy
  <chr>   <chr>      <dbl>         <dbl>
1 Germany 1960      2.41          69.3
2 Germany 1961      2.44          69.8
3 Germany 1962      2.47          70.0
4 Germany 1963      2.49          70.1
5 Germany 1964      2.49          70.7
6 Germany 1965      2.48          70.6
7 Germany 1966      2.44          70.8
8 Germany 1967      2.37          71.0
9 Germany 1968      2.28          70.6
10 Germany 1969      2.17          70.5
# ... with 102 more rows
# i Use `print(n = ...)` to see more rows
```

Steps:

1. Separate each individual number to a row (through `pivot_longer`)
2. Specify the variable name for each row/number (through `separate`)
3. Combine numbers that belong to the same observation (through `pivot_wider`)



In this lecture

- Data wrangling challenges
- Tidy, long, and wide
- **Joining & binding**
- Set operators in R



Joining tables

With different information from multiple data sources, it is common for user to join tables according to some unique identifiers (name, id, etc.)

Suppose we want to explore the relationship between population size and electoral votes of different states. We have the population size in the `murders` dataset, and the electoral votes from the `results_us_election_2016` dataset

```
> library(dslabs)
> data(murders)
> head(murders,2)
  state abb region population total
1 Alabama AL  South  4779736   135
2  Alaska AK   West   710231    19
> data(polls_us_election_2016)
> head(results_us_election_2016,2)
  state electoral_votes clinton trump others
1 California           55    61.7  31.6   6.7
2    Texas            38    43.2  52.2   4.5
```



left_join

Join the two tables by state using `left_join`.

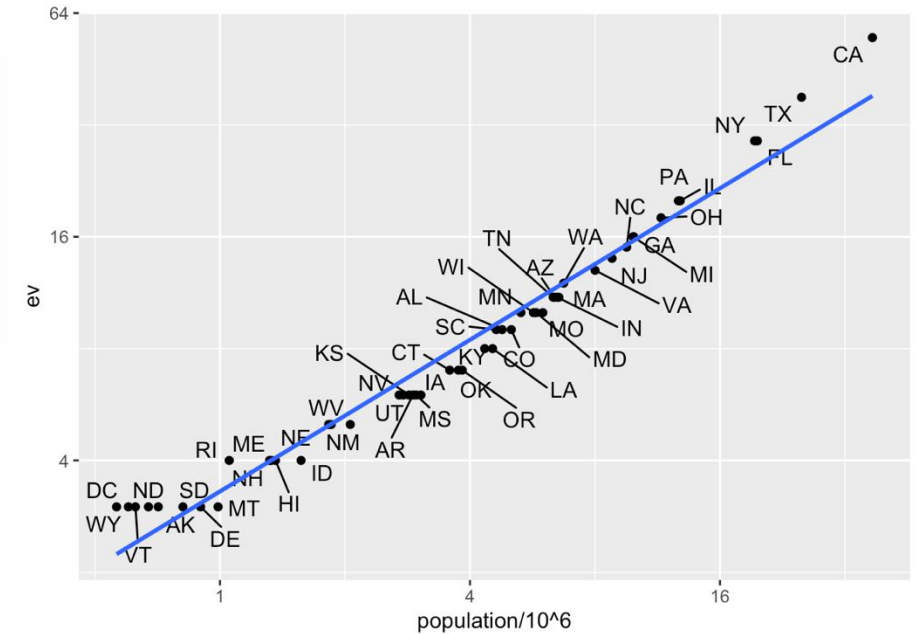
Remove the `others` column that is not needed and rename a column

```
> tab <- murders |>
+   left_join(results_us_election_2016, by = "state") |>
+   select(-others) |>
+   rename(ev = "electoral_votes")
> head(tab, n = 5)
```

	state	abb	region	population	total	ev	clinton	trump
1	Alabama	AL	South	4779736	135	9	34.4	62.1
2	Alaska	AK	West	710231	19	3	36.6	51.3
3	Arizona	AZ	West	6392017	232	11	45.1	48.7
4	Arkansas	AR	South	2915918	93	6	33.7	60.6
5	california	CA	West	37253956	1257	55	61.7	31.6


```
> library(ggrepel)
> tab |> ggplot(aes(population/10^6, ev, label = abb)) +
+ geom_point() +
+ geom_text_repel(max.overlaps = 30) +
+ scale_x_continuous(trans = "log2") +
+ scale_y_continuous(trans = "log2") +
+ geom_smooth(method = "lm", se = FALSE)
```

We see the relationship is close to linear with about 2 electoral votes for every million persons, but with very small states getting higher ratios



Handling unmatched rows

In practice, it is not always the case that each row in one table has a matching row in another. For this reason, we have several versions of join. To demonstrate this challenge, we use a subset of the two tables. We create the tables `tab_1` and `tab_2` so that they have some states in common but not all

```
> tab_1 <- slice(murders, 1:6) |> select(state, population)
> head(tab_1)
```

	state	population
1	Alabama	4779736
2	Alaska	710231
3	Arizona	6392017
4	Arkansas	2915918
5	California	37253956
6	Colorado	5029196

```
> tab_2 <- results_us_election_2016 |>
+ filter(state%in%c("Alabama", "Alaska", "Arizona",
+                   "California", "Connecticut", "Delaware")) |>
+ select(state, electoral_votes) |> rename(ev = electoral_votes)
> head(tab_2)
```

	state	ev
1	California	55
2	Arizona	11
3	Alabama	9
4	Connecticut	7
5	Alaska	3
6	Delaware	3



Different types of join functions

Suppose we want a table like `tab_1`, but adding any available electoral votes from `tab_2`. For this, we use `left_join` with `tab_1` as the first argument, and `tab_2` as the second. We specify which column to use to match with the `by` argument.

```
> left_join(tab_1, tab_2, by = "state")
  state population ev
1 Alabama    4779736 9
2  Alaska     710231 3
3  Arizona    6392017 11
4 Arkansas    2915918 NA
5 California  37253956 55
6  Colorado    5029196 NA
```

States seen in `tab_1` but not
`tab_2` are assigned NA



Different types of join functions

If we want a table like `tab_2`, but adding any available population from `tab_1`. For this, we use `right_join` with `tab_1` as the first argument and `tab_2` the second.

Notice that `right_join(tab_1, tab_2)` is equivalent to `left_join(tab_2, tab_1)`

```
> right_join(tab_1, tab_2, by = "state")
  state population ev
1  Alabama    4779736 9
2   Alaska     710231 3
3   Arizona    6392017 11
4 California   37253956 55
5 Connecticut         NA 7
6   Delaware         NA 3
```

States seen in `tab_2` but not `tab_1` are assigned NA



Different types of join functions

If we want to keep only the rows that have information in both tables, we use `inner_join`

```
> inner_join(tab_1, tab_2, by = "state")
  state population ev
1  Alabama    4779736 9
2   Alaska     710231 3
3  Arizona    6392017 11
4 California  37253956 55
```


Different types of join functions

If we want to keep all the rows from both tables and fill the missing parts with NAs, we use `full_join`

```
> full_join(tab_1, tab_2, by = "state")
```

	state	population	ev
1	Alabama	4779736	9
2	Alaska	710231	3
3	Arizona	6392017	11
4	Arkansas	2915918	NA
5	California	37253956	55
6	Colorado	5029196	NA
7	Connecticut	NA	7
8	Delaware	NA	3





Different types of join functions

The `semi_join` function lets us keep the part of first table that have matching rows in the second. It DOES NOT add the columns from the second table.

```
> semi_join(tab_1, tab_2, by = "state")
  state population
1  Alabama    4779736
2   Alaska     710231
3  Arizona    6392017
4 California  37253956
```



Different types of join functions

The function `anti_join` is the opposite of `semi_join`. It only keeps rows of the first table that don't have matches in the second table.

```
> anti_join(tab_1, tab_2, by = "state")  
  state population  
1 Arkansas    2915918  
2 Colorado    5029196
```

RELATIONAL DATA

Use a "**Mutating Join**" to join one table to columns from another, matching values with the rows that they correspond to. Each join retains a different combination of values from the tables.

A	B	C	D
a	t	1	3
b	u	2	2
c	v	3	NA

left_join(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ..., keep = FALSE, na_matches = "na") Join matching values from y to x.

A	B	C	D
a	t	1	3
b	u	2	2
d	w	NA	1

right_join(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ..., keep = FALSE, na_matches = "na") Join matching values from x to y.

A	B	C	D
a	t	1	3
b	u	2	2

inner_join(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ..., keep = FALSE, na_matches = "na") Join data. Retain only rows with matches.

A	B	C	D
a	t	1	3
b	u	2	2
c	v	3	NA
d	w	NA	1

full_join(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ..., keep = FALSE, na_matches = "na") Join data. Retain all values, all rows.

Use a "**Filtering Join**" to filter one table against the rows of another.

x		
A	B	C
a	t	1
b	u	2
c	v	3

y			
A	B	C	D
a	t	3	
b	u	2	
d	w	1	

+

A	B	C
a	t	1
b	u	2

semi_join(x, y, by = NULL, copy = FALSE, ..., na_matches = "na") Return rows of x that have a match in y. Use to see what will be included in a join.

A	B	C
c	v	3

anti_join(x, y, by = NULL, copy = FALSE, ..., na_matches = "na") Return rows of x that do not have a match in y. Use to see what will not be included in a join.

Use a "**Nest Join**" to inner join one table to another into a nested data frame.

A	B	C	y
a	t	1	<tibble [1x2]>
b	u	2	<tibble [1x2]>
c	v	3	<tibble [1x2]>

nest_join(x, y, by = NULL, copy = FALSE, keep = FALSE, name = NULL, ...) Join data, nesting matches from y in a single new data frame column.

binding

Unlike the join functions, the binding functions do not try to match by a variable, but instead simply combine datasets. If the datasets don't match by the appropriate dimensions, yield an error

```
> bind_cols(a = 1:3, b = 4:6)
# A tibble: 3 × 2
      a     b
  <int> <int>
1     1     4
2     2     5
3     3     6
```

```
> tab_1 <- tab[1:2,]
> tab_2 <- tab[3:4,]
> bind_rows(tab_1, tab_2)
  state abb region population total ev clinton trump
1 Alabama AL South  4779736  135  9   34.4  62.1
2 Alaska  AK  West   710231   19  3   36.6  51.3
3 Arizona AZ  West  6392017  232 11   45.1  48.7
4 Arkansas AR South 2915918   93  6   33.7  60.6
```

```
> tab_1 <- tab[, 1:3]
> tab_2 <- tab[, 4:6]
> tab_3 <- tab[, 7:8]
> new_tab <- bind_cols(tab_1, tab_2, tab_3)
> head(new_tab)
  state abb region population total ev clinton trump
1 Alabama AL South  4779736  135  9   34.4  62.1
2 Alaska  AK  West   710231   19  3   36.6  51.3
3 Arizona AZ  West  6392017  232 11   45.1  48.7
4 Arkansas AR South 2915918   93  6   33.7  60.6
5 California CA  West 37253956 1257 55   61.7  31.6
6 Colorado CO  West  5029196   65  9   48.2  43.3
```


binding

Be aware of the behavior when combining rows/columns from different tables

```
> bind_cols(murders[1:5, 1:3], results_us_election_2016[1:5, 1:3])
New names:
• `state` -> `state...1`
• `state` -> `state...4`
  state...1 abb region state...4 electoral_votes clinton
1   Alabama  AL  South California          55      61.7
2   Alaska   AK   West   Texas             38      43.2
3   Arizona  AZ   West   Florida            29      47.8
4   Arkansas AR   South  New York            29      59.0
5 California CA   West   Illinois           20      55.8
> bind_rows(murders[1:5, 1:3], results_us_election_2016[1:5, 1:3])
  state  abb region electoral_votes clinton
1  Alabama  AL  South             NA      NA
2  Alaska   AK   West             NA      NA
3  Arizona  AZ   West             NA      NA
4  Arkansas AR   South            NA      NA
5 California CA   West            NA      NA
6 California <NA> <NA>           55      61.7
7   Texas <NA> <NA>           38      43.2
8   Florida <NA> <NA>           29      47.8
9   New York <NA> <NA>           29      59.0
10  Illinois <NA> <NA>           20      55.8
```



In this lecture

- Data wrangling challenges
- Tidy, long, and wide
- Joining & binding
- **Set operators in R**



intersect

Another set of commands useful for combining datasets are the set operators.

```
> intersect(1:10, 6:15)
[1] 6 7 8 9 10
> intersect(c("a","b","c"), c("b","c","d"))
[1] "b" "c"
```

```
> tab_1 <- tab[1:5,]
> tab_2 <- tab[3:7,]
> dplyr::intersect(tab_1, tab_2)
```

	state	abb	region	population	total	ev	clinton	trump
1	Arizona	AZ	West	6392017	232	11	45.1	48.7
2	Arkansas	AR	South	2915918	93	6	33.7	60.6
3	California	CA	West	37253956	1257	55	61.7	31.6



union

```
> union(1:10, 6:15)
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
> union(c("a","b","c"), c("b","c","d"))
[1] "a" "b" "c" "d"
```

```
> tab_1 <- tab[1:5,]
> tab_2 <- tab[3:7,]
> dplyr::union(tab_1, tab_2)
```

	state	abb	region	population	total	ev	clinton	trump
1	Alabama	AL	South	4779736	135	9	34.4	62.1
2	Alaska	AK	West	710231	19	3	36.6	51.3
3	Arizona	AZ	West	6392017	232	11	45.1	48.7
4	Arkansas	AR	South	2915918	93	6	33.7	60.6
5	California	CA	West	37253956	1257	55	61.7	31.6
6	Colorado	CO	West	5029196	65	9	48.2	43.3
7	Connecticut	CT	Northeast	3574097	97	7	54.6	40.9



setdiff

(NOT symmetric) The set difference between the first and the second argument as:

```
set(ARG1) - set(ARG2)
```

```
> setdiff(1:10, 6:15)
[1] 1 2 3 4 5
> setdiff(6:15, 1:10)
[1] 11 12 13 14 15
```

```
> tab_1 <- tab[1:5,]
> tab_2 <- tab[3:7,]
> dplyr::setdiff(tab_1, tab_2)
```

	state	abb	region	population	total	ev	clinton	trump
1	Alabama	AL	South	4779736	135	9	34.4	62.1
2	Alaska	AK	West	710231	19	3	36.6	51.3



setequal

If the two sets are the same, regardless of order

```
> setequal(1:5, 1:6)
[1] FALSE
> setequal(1:5, 5:1)
[1] TRUE
```

```
> dplyr::setequal(tab_1, tab_2)
[1] FALSE
```



In this lecture

- Data wrangling challenges
- Tidy, long, and wide
 - `pivot_longer`, `pivot_wider`
 - `separate`
- Joining & binding
 - `left_join`, `right_join`, `inner_join`,
`full_join`, `semi_join`, `anti_join`
 - `bind_rows`, `bind_cols`
- Set operators in R
 - `intersect`, `union`, `setdiff`, `setequal`